

# A Formal Algorithmic Analysis of Binary Tree Data Structures: Traversal Strategies, View Projections, and Level-Order Processing

## Abstract

Trees represent a fundamental non-linear hierarchical data structure with applications spanning compiler design, database indexing, file system architectures, and network routing. This paper presents a rigorous formal treatment of binary tree structures, establishing definitions for core tree properties including depth, height, and level. We classify binary trees into three canonical forms: Proper (Full), Complete, and Perfect binary trees, and derive bounds on node counts at each level ( $2^k$  at level  $k$ , for a total of  $2^{h+1} - 1$  nodes in a perfect tree of height  $h$ ). We formalize the three recursive depth-first traversal strategies — Pre-order (Root-Left-Right), In-order (Left-Root-Right), and Post-order (Left-Right-Root) — providing complete implementations and dry-run execution traces. We then introduce the iterative Breadth-First (Level-Order) traversal using a FIFO queue, analyzing its  $O(N)$  time and  $O(W)$  space complexity where  $W$  is the maximum width of the tree. Finally, we formalize Left View and Right View projection algorithms as specialized level-order traversals that extract the first and last visible nodes at each depth level respectively. All algorithms are implemented in Java with formal complexity proofs and illustrative TikZ diagrams.

## Index Terms

linked list, deep copy, random pointer, hash map, node interleaving, Java, algorithm analysis, computational complexity



## 1 Introduction

While linear data structures such as arrays, linked lists, stacks, and queues store elements in sequential order, many real-world relationships are inherently hierarchical. The Tree data structure captures this hierarchy by organizing elements in a parent-child relationship. Trees are utilized across computer science:

- Compiler Design: Abstract Syntax Trees (ASTs) represent parsed program structure.
- Database Systems: B-Trees and B+ Trees provide efficient indexing for disk-based storage.
- File Systems: Directory structures are modeled as trees with the root directory at the top.
- Networking: Spanning trees prevent loops in network topologies.
- Artificial Intelligence: Decision trees and game trees guide search and classification.

This paper provides a formal algorithmic treatment of binary trees. Section 2 defines the core terminology and properties. Section 3 classifies binary tree types and derives node count bounds. Section 4 formalizes recursive depth-first traversal strategies with execution traces. Section 5 details level-order (BFS) traversal. Section 6 presents Left View and Right View algorithms.

## 2 Tree Fundamentals and Terminology

**Definition 1 (Tree).** A tree  $T = (V, E)$  is a connected, acyclic, undirected graph where  $V$  is a finite set of vertices (nodes) and  $E$  is a set of edges such that  $|E| = |V| - 1$ . One distinguished node  $r \in V$  is designated as the root.

**Definition 2 (Binary Tree).** A binary tree is a rooted tree in which every node has at most two children, referred to as the left child and the right child. Formally, each node  $v$  has a left subtree  $T_L(v)$  and a right subtree  $T_R(v)$ , either of which may be empty ( $\emptyset$ ).

Key terminology for trees is defined as follows:

- Root: The topmost node of the tree with no parent. Every tree has exactly one root.
- Parent & Child: If node  $u$  has an edge to node  $v$  where  $u$  is closer to the root, then  $u$  is the parent of  $v$ , and  $v$  is a child of  $u$ .
- Leaf Node: A node with no children, i.e., both  $T_L(v) = \emptyset$  and  $T_R(v) = \emptyset$ .
- Internal Node: A node that has at least one child.
- Edge: A connection between a parent and a child. A tree with  $N$  nodes has exactly  $N - 1$  edges.
- Subtree: The tree rooted at any node  $v$  consists of  $v$  and all its descendants.

- Depth of a Node: The number of edges on the path from the root to that node. The root has depth 0.
- Height of a Node: The number of edges on the longest path from that node to a leaf descendant.
- Height of the Tree: The height of the root node, i.e., the longest path from root to any leaf.
- Level  $k$ : The set of all nodes at depth  $k$ .

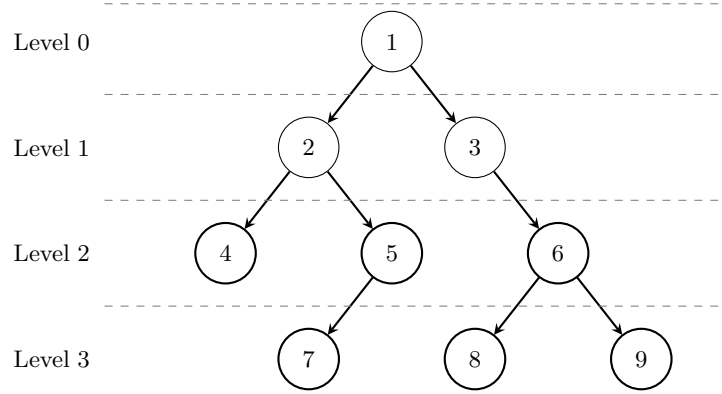


Fig. 1. A binary tree with 9 nodes. The root (node 1) has depth 0 and height 3. Node 7, 8, 9, and 4 are leaf nodes at levels 3 and 2.

## 2.1 Binary Tree Node Representation

We represent a binary tree node in Java as follows:

```

class Node {
    int data;
    Node left;
    Node right;

    Node(int data) {
        this.data = data;
        this.left = null;
        this.right = null;
    }
}

```

Each Node object is allocated on the heap and contains three fields: an integer payload (data), and two reference pointers to its left and right children. When a child does not exist, the reference is null.

## 3 Classification of Binary Trees

Binary trees are classified into three canonical forms based on their structural constraints:

### 3.1 Full (Proper/Strict) Binary Tree

**Definition 3 (Full Binary Tree).** A binary tree is full (or proper or strict) if every internal node has exactly 2 children. Equivalently, no node has exactly one child.

### 3.2 Complete Binary Tree

**Definition 4 (Complete Binary Tree).** A binary tree of height  $h$  is complete if all levels  $0, 1, \dots, h - 1$  are completely filled, and the last level  $h$  is filled from left to right without gaps.

Complete binary trees are the basis for the binary heap data structure and can be efficiently stored in an array using index arithmetic: for a node at index  $i$  (0-based), its left child is at  $2i + 1$  and right child at  $2i + 2$ .

### 3.3 Perfect Binary Tree

**Definition 5 (Perfect Binary Tree).** A binary tree of height  $h$  is perfect if all internal nodes have exactly 2 children and all leaf nodes are at the same level  $h$ .

**Theorem 6 (Node Count in a Perfect Binary Tree).** A perfect binary tree of height  $h$  has exactly  $2^{h+1} - 1$  total nodes.

Proof. At level  $k$  ( $0 \leq k \leq h$ ), there are exactly  $2^k$  nodes. The total number of nodes is:

$$N = \sum_{k=0}^h 2^k = 2^{h+1} - 1 \quad (1)$$

This follows directly from the geometric series sum formula  $\sum_{k=0}^n r^k = \frac{r^{n+1}-1}{r-1}$  with  $r = 2$ .  $\square$

Corollary 7. The number of leaf nodes in a perfect binary tree of height  $h$  is  $2^h$ , which equals the number of internal nodes plus one.

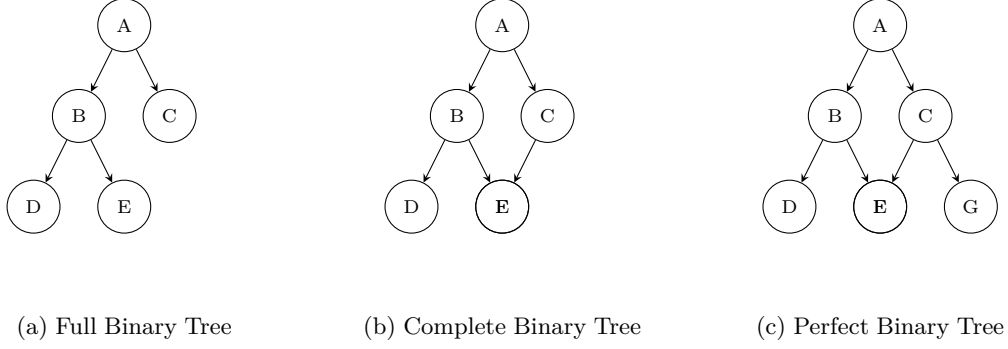


Fig. 2. Classification of binary trees. (a) Full: every node has 0 or 2 children. (b) Complete: all levels filled except possibly the last, which is filled left-to-right. (c) Perfect: all internal nodes have 2 children and all leaves are at the same level.

Table 1 summarizes the node count bounds for each classification.

TABLE 1  
Node Count Bounds for Binary Tree Classifications (height  $h$ ).

| Property             | Full          | Complete      | Perfect       |
|----------------------|---------------|---------------|---------------|
| Min Nodes            | $2h + 1$      | $2^h$         | $2^{h+1} - 1$ |
| Max Nodes            | $2^{h+1} - 1$ | $2^{h+1} - 1$ | $2^{h+1} - 1$ |
| Leaves at level $h$  | $\geq 1$      | $\geq 1$      | $2^h$         |
| Array Representation | No            | Yes           | Yes           |

## 4 Recursive Depth-First Traversal Strategies

Tree traversal is the process of visiting every node in the tree exactly once in a defined order. The three canonical depth-first traversal strategies are defined by the position at which the root is processed relative to its subtrees.

### 4.1 Pre-order Traversal (Root $\rightarrow$ Left $\rightarrow$ Right)

In pre-order traversal, we first process the current node, then recursively traverse the left subtree, and finally recursively traverse the right subtree.

```
void preorder(Node root) {
    if (root == null) {
        return; // Base case: empty subtree
    }
    System.out.print(root.data + " "); // Process root
    preorder(root.left);               // Recurse left
    preorder(root.right);              // Recurse right
}
```

Pre-order applications: Creating a copy of the tree, serializing tree structure for reconstruction, and generating prefix expressions from expression trees.

#### 4.2 In-order Traversal (Left $\rightarrow$ Root $\rightarrow$ Right)

In in-order traversal, we first recursively traverse the left subtree, then process the current node, and finally recursively traverse the right subtree.

```
void inorder(Node root) {
    if (root == null) {
        return; // Base case: empty subtree
    }
    inorder(root.left);      // Recurse left
    System.out.print(root.data + " "); // Process root
    inorder(root.right);     // Recurse right
}
```

In-order applications: For Binary Search Trees (BSTs), in-order traversal produces elements in sorted (non-decreasing) order, making it useful for validation and sorted output.

#### 4.3 Post-order Traversal (Left $\rightarrow$ Right $\rightarrow$ Root)

In post-order traversal, we first recursively traverse the left subtree, then recursively traverse the right subtree, and finally process the current node.

```
void postorder(Node root) {
    if (root == null) {
        return; // Base case: empty subtree
    }
    postorder(root.left);    // Recurse left
    postorder(root.right);   // Recurse right
    System.out.print(root.data + " "); // Process root
}
```

Post-order applications: Deleting a tree (children must be freed before their parent), evaluating postfix expressions, and computing directory sizes in file systems.

#### 4.4 Execution Trace (Dry Run)

Consider the binary tree from Figure 1 with nodes  $\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$ . We trace each traversal order:

TABLE 2  
Traversal Output for the Binary Tree in Figure 1.

| Traversal Strategy           | Output Sequence           |
|------------------------------|---------------------------|
| Pre-order (Root-Left-Right)  | 1, 2, 4, 5, 7, 3, 6, 8, 9 |
| In-order (Left-Root-Right)   | 4, 2, 7, 5, 1, 3, 8, 6, 9 |
| Post-order (Left-Right-Root) | 4, 7, 5, 2, 8, 9, 6, 3, 1 |

#### 4.5 Complexity Analysis

**Theorem 8.** All three recursive depth-first traversals run in  $O(N)$  time and  $O(H)$  auxiliary space, where  $N$  is the number of nodes and  $H$  is the height of the tree.

**Proof.** Each node is visited exactly once across all recursive calls, giving  $O(N)$  total work. The space complexity is determined by the maximum depth of the recursion call stack. In the worst case (a skewed tree where  $H = N - 1$ ), the space complexity is  $O(N)$ . In the best case (a balanced tree where  $H = \lfloor \log_2 N \rfloor$ ), the space complexity is  $O(\log N)$ .  $\square$

### 5 Level-Order Traversal (Breadth-First Search)

Unlike depth-first strategies that dive deep into subtrees, level-order traversal (also known as Breadth-First Search, BFS) visits all nodes at depth  $k$  before visiting any node at depth  $k + 1$ . This is achieved using a FIFO queue.

#### 5.1 Algorithm

- 1) Initialize a queue  $Q$  and enqueue the root node.
- 2) While  $Q$  is not empty:
  - a) Dequeue a node  $c$  from the front of  $Q$ .
  - b) Process (print)  $c$ .
  - c) If  $c.\text{left} \neq \text{null}$ , enqueue  $c.\text{left}$ .
  - d) If  $c.\text{right} \neq \text{null}$ , enqueue  $c.\text{right}$ .

## 5.2 Java Implementation

```
import java.util.Queue;
import java.util.LinkedList;

void levelOrder(Node root) {
    if (root == null) return;

    Queue<Node> q = new LinkedList<>();
    q.add(root);

    while (!q.isEmpty()) {
        Node curr = q.remove();
        System.out.print(curr.data + " ");

        if (curr.left != null) {
            q.add(curr.left);
        }
        if (curr.right != null) {
            q.add(curr.right);
        }
    }
}
```

## 5.3 Execution Trace

For the tree in Figure 1, the level-order traversal proceeds as follows:

TABLE 3  
Level-Order BFS Execution Trace.

| Step | Dequeued Node | Queue State After         | Output So Far             |
|------|---------------|---------------------------|---------------------------|
| 1    | 1             | $\langle 2, 3 \rangle$    | 1                         |
| 2    | 2             | $\langle 3, 4, 5 \rangle$ | 1, 2                      |
| 3    | 3             | $\langle 4, 5, 6 \rangle$ | 1, 2, 3                   |
| 4    | 4             | $\langle 5, 6 \rangle$    | 1, 2, 3, 4                |
| 5    | 5             | $\langle 6, 7 \rangle$    | 1, 2, 3, 4, 5             |
| 6    | 6             | $\langle 7, 8, 9 \rangle$ | 1, 2, 3, 4, 5, 6          |
| 7    | 7             | $\langle 8, 9 \rangle$    | 1, 2, 3, 4, 5, 6, 7       |
| 8    | 8             | $\langle 9 \rangle$       | 1, 2, 3, 4, 5, 6, 7, 8    |
| 9    | 9             | $\langle \rangle$         | 1, 2, 3, 4, 5, 6, 7, 8, 9 |

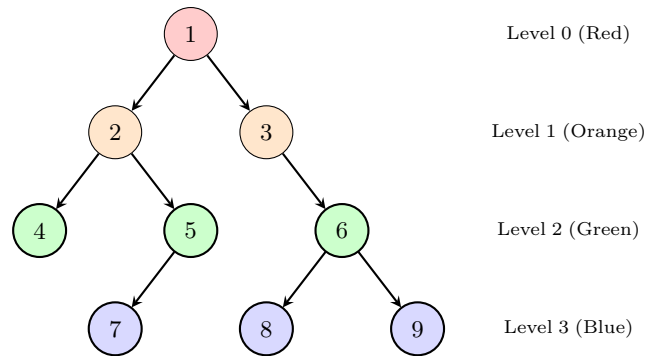


Fig. 3. Level-order (BFS) traversal visits nodes level-by-level. Colors indicate the order: Level 0 (root) is processed first, then Level 1, then Level 2, and finally Level 3.

## 5.4 Complexity Analysis

Theorem 9. Level-order traversal runs in  $O(N)$  time and  $O(W)$  auxiliary space, where  $W$  is the maximum width (maximum number of nodes at any single level) of the tree.

Proof. Each of the  $N$  nodes is enqueued and dequeued exactly once, each operation taking  $O(1)$  time. Thus the total time is  $O(N)$ . The queue holds at most  $W$  nodes simultaneously, where  $W$  is the width of the widest level. For a perfect binary tree of height  $h$ ,  $W = 2^h = \frac{N+1}{2} = O(N)$  in the worst case. For a skewed tree,  $W = 1$ , giving  $O(1)$  space.  $\square$

## 6 Binary Tree View Projections

View projections extract the visible nodes when a binary tree is observed from a particular direction. These are computed using modified level-order traversals.

### 6.1 Left View

The Left View of a binary tree consists of the first node visible at each level when the tree is viewed from the left side. Formally, it is the set of nodes  $\{v_k : v_k \text{ is the first node at level } k\}$  for  $k = 0, 1, \dots, h$ .

#### 6.1.1 Algorithm

We perform a level-order traversal and, for each level, record only the first node encountered:

```
import java.util.Queue;
import java.util.LinkedList;
import java.util.ArrayList;
import java.util.List;

List<Integer> leftView(Node root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;

    Queue<Node> q = new LinkedList<>();
    q.add(root);

    while (!q.isEmpty()) {
        int levelSize = q.size();

        for (int i = 0; i < levelSize; i++) {
            Node curr = q.remove();

            // First node of this level is the left view node
            if (i == 0) {
                result.add(curr.data);
            }

            if (curr.left != null) q.add(curr.left);
            if (curr.right != null) q.add(curr.right);
        }
    }
    return result;
}
```

### 6.2 Right View

The Right View of a binary tree consists of the last node visible at each level when the tree is viewed from the right side. Formally, it is the set of nodes  $\{v_k : v_k \text{ is the last node at level } k\}$  for  $k = 0, 1, \dots, h$ .

#### 6.2.1 Algorithm

We perform the same level-order traversal but record the last node at each level:

```
List<Integer> rightView(Node root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;

    Queue<Node> q = new LinkedList<>();
    q.add(root);

    while (!q.isEmpty()) {
```

```

int levelSize = q.size();

for (int i = 0; i < levelSize; i++) {
    Node curr = q.remove();

    // Last node of this level is the right view node
    if (i == levelSize - 1) {
        result.add(curr.data);
    }

    if (curr.left != null) q.add(curr.left);
    if (curr.right != null) q.add(curr.right);
}
return result;
}

```

### 6.3 View Projection Trace

For the binary tree in Figure 1:

TABLE 4  
Left View and Right View Projections.

| Level | Nodes at Level | Left View | Right View |
|-------|----------------|-----------|------------|
| 0     | {1}            | 1         | 1          |
| 1     | {2, 3}         | 2         | 3          |
| 2     | {4, 5, 6}      | 4         | 6          |
| 3     | {7, 8, 9}      | 7         | 9          |

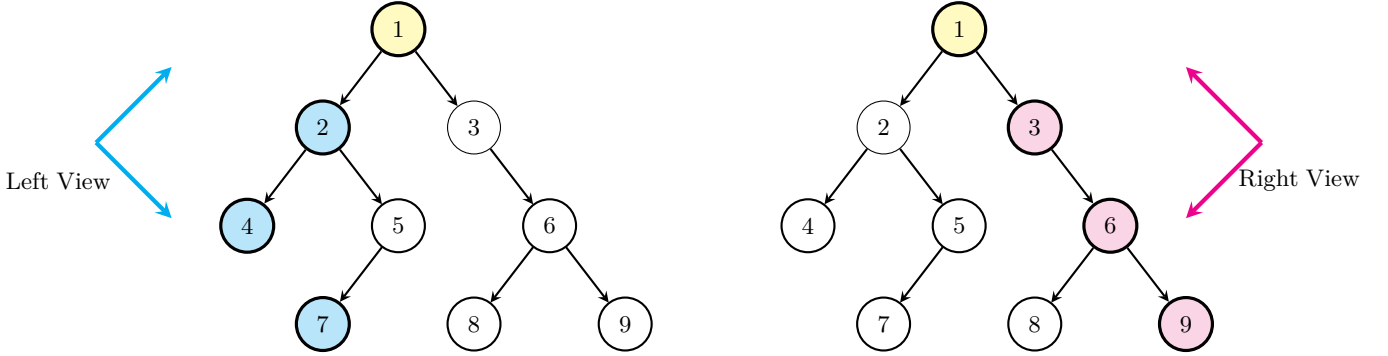


Fig. 4. Left View (cyan highlighted nodes: 1, 2, 4, 7) and Right View (magenta highlighted nodes: 1, 3, 6, 9) of the binary tree.

### 6.4 Complexity Analysis

Theorem 10. Both Left View and Right View algorithms run in  $O(N)$  time and  $O(W)$  auxiliary space.

Proof. The algorithms perform a standard level-order traversal, visiting each of the  $N$  nodes exactly once. The conditional check ( $i == 0$  for Left View,  $i == \text{levelSize} - 1$  for Right View) adds only  $O(1)$  overhead per node. The queue stores at most  $W$  nodes at any time, where  $W$  is the maximum tree width. Therefore, the time complexity is  $O(N)$  and the space complexity is  $O(W)$ .  $\square$

## 7 Summary of Complexity Bounds

Table 5 consolidates the time and space complexities of all algorithms presented in this paper.

TABLE 5  
Consolidated Complexity Analysis of Binary Tree Algorithms.

| Algorithm            | Time   | Aux. Space (Balanced) | Aux. Space (Skewed) |
|----------------------|--------|-----------------------|---------------------|
| Pre-order Traversal  | $O(N)$ | $O(\log N)$           | $O(N)$              |
| In-order Traversal   | $O(N)$ | $O(\log N)$           | $O(N)$              |
| Post-order Traversal | $O(N)$ | $O(\log N)$           | $O(N)$              |
| Level-order (BFS)    | $O(N)$ | $O(N/2) = O(N)$       | $O(1)$              |
| Left View            | $O(N)$ | $O(N)$                | $O(1)$              |
| Right View           | $O(N)$ | $O(N)$                | $O(1)$              |

## 8 Conclusion

This paper has presented a formal algorithmic treatment of binary tree data structures. We established precise definitions for core tree terminology, classified binary trees into Full, Complete, and Perfect forms, and derived the node count bound  $2^{h+1} - 1$  for perfect trees using geometric series. We formalized the three recursive depth-first traversal strategies (Pre-order, In-order, Post-order), providing complete Java implementations with dry-run execution traces. We introduced level-order (BFS) traversal as an iterative approach using a FIFO queue, proving its  $O(N)$  time and  $O(W)$  space bounds. Finally, we presented Left View and Right View algorithms as specialized level-order traversals that project the first and last visible nodes at each depth level respectively. The interplay between recursive depth-first and iterative breadth-first approaches highlights the fundamental trade-offs in tree algorithm design: recursion leverages the call stack for implicit state management at the cost of  $O(H)$  stack space, while iteration with explicit queues provides  $O(W)$  space behavior that adapts to tree shape.